

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №4
по курсу «Программирование графических процессоров»**

Работа с матрицами. Метод Гаусса.

Выполнил: Орусский В.Р.

Группа: М8О-406Б-21

Преподаватель: А.Ю. Морозов

Москва, 2025

Условие

Цель работы: Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof (npu для более современных видеокарт).

Вариант 1. Вычисление детерминанта матрицы.

Входные данные. На первой строке задано число n -- размер матрицы. В следующих n строках, записано по n вещественных чисел - элементы матрицы. $n \leq 10^4$.

Программное и аппаратное обеспечение

GPU:

- Название NVIDIA GeForce GTX 1660 TI
- Compute capability: 7.5
- Графическая память: 2147155968
- Разделяемая память: 49152
- Константная память: 65536
- Количество регистров на блок: 32
- Максимальное количество нитей: (1024, 1024, 64)
- Максимальное количество блоков: (2147483647, 65535, 65535)
- Количество мультипроцессоров: 24

Сведения о системе:

- Процессор: Intel(R) Core(TM) i5-10600KF CPU @ 4.5000MHz
- ОЗУ: 16 ГБ
- SSD M2 1 ТБ

Программное обеспечение:

- OS: Windows 10 + WSL (Ubuntu 20.04)
- IDE: Visual Studio Code
- Компилятор: nvcc

Метод решения

Программа реализует метод вычисления определителя матрицы с использованием LU-разложения с выбором ведущего элемента на GPU. Транспонирование матрицы происходит на GPU. Далее выполняется LU-разложение с частичным выбором (замена строки, если абсолютный максимум находится не на диагонали). Затем вычисляются элементы нижней треугольной матрицы L, где диагональные элементы L принимаются равными 1, а остальные элементы в столбце под текущей строкой вычисляются как отношение соответствующих элементов матрицы к ведущему элементу. После чего определитель вычисляется на хосте, как произведение элементов на диагонали.

Описание программы

Вся программа описана в одном файле main.cu. Есть следующие функции:

- *gpu_transpose*: Транспонирует матрицу размером $\text{size} \times \text{size}$, меняя местами элементы симметрично относительно главной диагонали.
- *gpu_swap*: Меняет местами строки для реализации частичного выбора ведущего элемента.
- *gpu_compute_L*: Вычисляет элементы столбца матрицы L для текущей строки `curr_row`.
- *gpu_modify_matrix*: Обновляет матрицу, вычитая произведения элементов L и текущего столбца для формирования верхней треугольной матрицы U.
- *calculateDet*: Вычисляет определитель матрицы как произведение диагональных элементов матриц U и L с учетом знака перестановок.

Результаты

1. Конфигурация $\lll \text{dim3}(2, 2), \text{dim3}(4, 4) \ggg$

n	Время (в мс)
10	7.64333
100	56.6798
1000	42849.5
10000	Долго

2. Конфигурация $\lll \text{dim3}(4, 4), \text{dim3}(8, 8) \ggg$

n	Время (в мс)
10	6.11274
100	29.3813
1000	11266.5
10000	Долго

3. Конфигурация $\lll \text{dim3}(8, 8), \text{dim3}(16, 16) \ggg$

n	Время (в мс)
10	7.68016
100	21.4262
1000	3153.46
10000	Долго

4. Конфигурация $\lll \text{dim3}(128, 128), \text{dim3}(256, 256) \ggg$

n	Время (в мс)
10	7.0103
100	21.9722
1000	631.305
10000	78633.9

5.CPU

n	Время (в мс)
10	0.09
100	5.23
1000	3242.42
10000	Долго

Исследование производительности

Исследование производительности было принято производить с помощью утилиты nvc, поскольку nvprof является устаревшей утилитой и не поддерживается на большинстве современных видеокарт / версий драйверов. Стоит отметить, что данные были сделаны на RTX 4080.

Вывод профайлера для `gpu_compute_L` и `updateMatrix`:

```
gpu_compute_L(double *, double *, int, int) (256, 1, 1)x(256, 1, 1), Context 1, Stream 7, Device 0, CC 8.9
Section: GPU Speed Of Light Throughput
-----
Metric Name      Metric Unit Metric Value
-----
DRAM Frequency   Ghz          11.01
SM Frequency      Ghz          2.25
Elapsed Cycles    cycle        6397
Memory Throughput %          1.27
DRAM Throughput   %          0.45
Duration          us          2.85
L1/TEX Cache Throughput %      5.48
L2 Cache Throughput %      1.27
SM Active Cycles  cycle       935.62
Compute (SM) Throughput %      0.95
-----

Section: Launch Statistics
-----
Metric Name      Metric Unit Metric Value
-----
Block Size              256
Function Cache Configuration CachePreference
Grid Size              256
Registers Per Thread    register/thread 36
Shared Memory Configuration Size Kbyte        16.38
Driver Shared Memory Per Block Kbyte/block   1.02
Dynamic Shared Memory Per Block byte/block        0
Static Shared Memory Per Block byte/block          0
# SMs                  SM            80
Stack Size             thread         1024
Threads                thread         65536
# TPCs                 40
Enabled TPC IDs        all
Uses Green Context      0
Waves Per SM           0.53
-----

Section: Occupancy
-----
Metric Name      Metric Unit Metric Value
-----
Block Limit SM          block         24
Block Limit Registers    block          6
Block Limit Shared Mem   block         16
Block Limit Warps        block          6
Theoretical Active Warps per SM warp           48
Theoretical Occupancy    %            100
Achieved Occupancy       %            44.86
Achieved Active Warps Per SM warp           21.54
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name      Metric Unit Metric Value
-----
Average DRAM Active Cycles cycle          140
Total DRAM Elapsed Cycles cycle       250880
Average L1 Active Cycles cycle       935.62
Total L1 Elapsed Cycles cycle     446264
Average L2 Active Cycles cycle     1060.97
Total L2 Elapsed Cycles cycle     173536
Average SM Active Cycles cycle       935.62
Total SM Elapsed Cycles cycle     446264
Average SMSP Active Cycles cycle      918.58
Total SMSP Elapsed Cycles cycle     1785056
-----
```

gpu_modify_matrix(double *, double *, int, int) (256, 1, 1)x(256, 1, 1), Context 1, Stream 7, Device 0, CC 8.9
Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	11.01
SM Frequency	Ghz	2.26
Elapsed Cycles	cycle	5284
Memory Throughput	%	1.13
DRAM Throughput	%	0.49
Duration	us	2.34
L1/TEX Cache Throughput	%	5.32
L2 Cache Throughput	%	0.63
SM Active Cycles	cycle	969.95
Compute (SM) Throughput	%	1.82

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		256
Function Cache Configuration	CachePreferNone	
Grid Size		256
Registers Per Thread	register/thread	24
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	80
Stack Size		1024
Threads	thread	65536
# TPCs		40
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		0.53

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	24
Block Limit Registers	block	10
Block Limit Shared Mem	block	16
Block Limit Warps	block	6
Theoretical Active Warps per SM	warp	48
Theoretical Occupancy	%	100
Achieved Occupancy	%	40.71
Achieved Active Warps Per SM	warp	19.54

Section: GPU and Memory Workload Distribution

Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	126
Total DRAM Elapsed Cycles	cycle	205824
Average L1 Active Cycles	cycle	969.95
Total L1 Elapsed Cycles	cycle	365976
Average L2 Active Cycles	cycle	228.19
Total L2 Elapsed Cycles	cycle	143520
Average SM Active Cycles	cycle	969.95
Total SM Elapsed Cycles	cycle	365976
Average SMSP Active Cycles	cycle	943.06
Total SMSP Elapsed Cycles	cycle	1463904

gpu_swap(double *, int, int, int) (256, 1, 1)x(256, 1, 1), Context 1, Stream 7, Device 0, CC 8.9

Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	11.01
SM Frequency	Ghz	2.24
Elapsed Cycles	cycle	5240
Memory Throughput	%	1.24
DRAM Throughput	%	0.30
Duration	us	2.34
L1/TEX Cache Throughput	%	5.64
L2 Cache Throughput	%	0.49
SM Active Cycles	cycle	910.35
Compute (SM) Throughput	%	1.29

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		256
Function Cache Configuration	CachePreferNone	
Grid Size		256
Registers Per Thread	register/thread	32
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	80
Stack Size		1024
Threads	thread	65536
# TPCs		40
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		0.53

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	24
Block Limit Registers	block	8
Block Limit Shared Mem	block	16
Block Limit Warps	block	6
Theoretical Active Warps per SM	warp	48
Theoretical Occupancy	%	100
Achieved Occupancy	%	60.64
Achieved Active Warps Per SM	warp	29.11

Section: GPU and Memory Workload Distribution

Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	78
Total DRAM Elapsed Cycles	cycle	205824
Average L1 Active Cycles	cycle	910.35
Total L1 Elapsed Cycles	cycle	330856
Average L2 Active Cycles	cycle	187.53
Total L2 Elapsed Cycles	cycle	142240
Average SM Active Cycles	cycle	910.35
Total SM Elapsed Cycles	cycle	330856
Average SMSP Active Cycles	cycle	920.12
Total SMSP Elapsed Cycles	cycle	1323424

```

gpu_transpose(double *, int) (256, 1, 1)x(256, 1, 1), Context 1, Stream 7, Device 0, CC 8.9
Section: GPU Speed Of Light Throughput
-----
Metric Name          Metric Unit Metric Value
-----
DRAM Frequency       Ghz          10.82
SM Frequency          Ghz          2.23
Elapsed Cycles        cycle         5060
Memory Throughput     %           1.21
DRAM Throughput       %           0.43
Duration              us           2.27
L1/TEX Cache Throughput %           5.62
L2 Cache Throughput   %           0.65
SM Active Cycles      cycle        917.34
Compute (SM) Throughput %           1.97
-----

Section: Launch Statistics
-----
Metric Name          Metric Unit Metric Value
-----
Block Size           256
Function Cache Configuration CachePreferNone
Grid Size            256
Registers Per Thread register/thread 28
Shared Memory Configuration Size Kbyte      16.38
Driver Shared Memory Per Block Kbyte/block 1.02
Dynamic Shared Memory Per Block byte/block   0
Static Shared Memory Per Block byte/block   0
# SMs                SM              80
Stack Size           1024
Threads              thread         65536
# TPCs               40
Enabled TPC IDs      all
Uses Green Context   0
Waves Per SM         0.53
-----

Section: Occupancy
-----
Metric Name          Metric Unit Metric Value
-----
Block Limit SM       block          24
Block Limit Registers block           8
Block Limit Shared Mem block          16
Block Limit Warps    block           6
Theoretical Active Warps per SM warp         48
Theoretical Occupancy %            100
Achieved Occupancy   %            41.76
Achieved Active Warps Per SM warp        20.04
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name          Metric Unit Metric Value
-----
Average DRAM Active Cycles cycle         106
Total DRAM Elapsed Cycles cycle       196608
Average L1 Active Cycles cycle        917.34
Total L1 Elapsed Cycles cycle       340588
Average L2 Active Cycles cycle        354.59
Total L2 Elapsed Cycles cycle       137376
Average SM Active Cycles cycle        917.34
Total SM Elapsed Cycles cycle       340588
Average SMSP Active Cycles cycle        938.55
Total SMSP Elapsed Cycles cycle     1362352
-----

```

Выводы

Вспомнил алгоритм разложения LU матрицы, познакомился с библиотекой Thrust, изучил профилировщики для CUDA nvprof и ncu. В ходе изучения первого профилировщика, пришёл к выводу о его неактуальности, пришлось использовать новый (ncu). По тестам видно, что CPU выигрывает во времени на малых входных данных, однако, уже начиная с размерности матрицы в 1000, GPU справляется с задачей быстрее, что отражает идею крупных вычислений через видеокарту.